

Build With AI — Personalized Implementation Playbook

Core Idea

Build With AI is a long-term learning, portfolio, and content system.

The goal is not to chase viral YouTube videos.

The goal is to become someone who can build real AI-powered systems, document the process publicly, and use those projects to qualify for stronger roles in:

- AI Engineering
- Applied AI Development
- Data Engineering
- Data Science
- Automation Engineering
- Backend Development
- Technical Product / Solutions Engineering

The rule:

Build real systems first. Content comes from the build.

Every project should produce:

- GitHub repository
- Portfolio case study
- YouTube video
- 5 TikTok / YouTube Shorts
- LinkedIn post
- Resume bullets
- Lessons learned document

The Long-Term Skill Stack

By following this roadmap, you gradually build these skill layers:

Layer 1 — Developer Foundations

- Python

- Git / GitHub
- APIs
- JSON
- environment variables
- project structure
- testing basics
- documentation

Layer 2 — AI Application Development

- LLM APIs
- prompt engineering
- structured outputs
- embeddings
- RAG
- vector databases
- AI evaluation

Layer 3 — Automation Engineering

- schedulers
- background jobs
- webhooks
- email automation
- data ingestion
- workflow orchestration

Layer 4 — Data / ML Engineering

- SQL
- PostgreSQL
- data pipelines
- feature engineering
- ML training
- MLflow
- model serving
- dashboards

Layer 5 — Agentic Systems

- tool calling
- ReAct patterns
- LangGraph
- multi-agent workflows
- agent memory
- planning and execution loops

Layer 6 — Production Systems

- FastAPI
 - authentication
 - Docker
 - CI/CD
 - observability
 - deployment
 - user accounts
 - payments
 - SaaS architecture
-

Project Progression Overview

Phase 1 — Foundations

Goal: Ship small but useful AI tools.

Projects:

1. Build Log AI
2. Smart Document Q&A System
3. AI Data Insight Assistant

Phase 2 — Automation

Goal: Build workflows that save time.

Projects:

1. AI Content Repurposing Pipeline
2. AI Job Hunter CRM
3. AI Email Triage Assistant

Phase 3 — Data & Intelligence

Goal: Move toward data science, analytics engineering, and ML systems.

Projects:

1. Market / Business Intelligence Monitor
2. ML Churn Prediction API
3. Natural Language to SQL Analytics Engine

Phase 4 — AI Agents

Goal: Build systems that can plan, use tools, and complete multi-step tasks.

Projects:

1. Autonomous Research Agent
2. Customer Support Agent System
3. AI Code Review / PR Assistant

Phase 5 — Production-Ready Systems

Goal: Build real software products with users, auth, deployment, and reliability.

Projects:

1. Full-Stack AI SaaS Starter
2. Visual Automation Platform

PHASE 1 — FOUNDATIONS

Project 1 — Build Log AI

Problem Being Solved

When building projects, it is hard to consistently summarize progress, write LinkedIn updates, create portfolio notes, and explain what you built.

Build Log AI reads your GitHub activity and turns it into clean weekly updates.

Why This Is Project 1

This project supports every future project.

It becomes your proof-of-work engine.

Technical Skills Learned

- GitHub API
- Python requests
- API authentication
- JSON parsing
- data aggregation
- LLM summarization

- Streamlit UI
- Markdown export
- basic deployment

Technologies Used

- Python
- GitHub API
- Streamlit
- OpenAI or Claude API
- SQLite
- Pandas
- Python-dotenv

Project Architecture

GitHub Username / Repo Input

↓

GitHub API Fetcher

↓

Commit + PR Data Parser

↓

Activity Aggregator

↓

LLM Summary Generator

↓

Streamlit Dashboard

↓

Export:

- LinkedIn post
- Weekly dev log
- YouTube update script
- TikTok hooks
- Portfolio notes

MVP Scope

- Input GitHub username and repository
- Fetch commits from last 7 or 30 days
- Summarize commits into human-readable progress
- Generate:
 - weekly dev summary
 - LinkedIn post
 - YouTube progress update
 - 5 TikTok hooks
- Export as Markdown

Stretch Goals

- Pull multiple repos
- Add GitHub contribution chart
- Generate resume bullets
- Generate portfolio case study draft
- Deploy publicly
- Add scheduled weekly email report

GitHub Repository Structure

```
build-log-ai/  
├─ app.py  
├─ src/  
│   ├── github_client.py  
│   ├── summarizer.py  
│   ├── formatter.py  
│   ├── database.py  
│   └─ export.py  
├─ prompts/  
│   ├── linkedin_post.txt  
│   ├── weekly_summary.txt  
│   └─ short_hooks.txt  
├─ data/  
│   └─ sample_commits.json  
├─ tests/  
│   └─ test_formatter.py  
├─ .env.example  
├─ requirements.txt  
└─ README.md
```

Portfolio Description

Built an AI-powered developer activity dashboard that connects to the GitHub API, analyzes recent commits, and generates weekly progress summaries, LinkedIn posts, portfolio notes, and short-form content ideas.

Resume Bullet Points

- Built an AI developer productivity tool using Python, Streamlit, and the GitHub API to summarize commit activity into structured weekly reports and social content.
- Integrated LLM-generated summaries with Markdown export, reusable prompt templates, and a clean dashboard interface for public proof-of-work tracking.

YouTube Video Ideas

- I Built an AI That Turns My GitHub Commits Into LinkedIn Posts

- How I Built My First AI Developer Tool
- Building My Proof-of-Work System From Scratch

TikTok / Shorts Ideas

- AI turned my GitHub commits into a LinkedIn post
- My project now writes its own progress update
- GitHub API in 60 seconds
- How I document projects without wasting time
- The first tool in my Build With AI journey

Estimated Difficulty

Beginner+

Estimated Completion Time

1–2 weeks

Deployment Recommendation

Deploy with Streamlit Cloud.

Project 2 — Smart Document Q&A System

Problem Being Solved

People have PDFs, notes, reports, and course materials but cannot easily ask questions across them with accurate citations.

This project teaches the foundation of RAG.

Technical Skills Learned

- text extraction
- chunking
- embeddings
- vector search
- RAG
- citations
- conversation memory
- prompt grounding
- hallucination reduction

Technologies Used

- Python
- Streamlit
- ChromaDB or FAISS
- OpenAI embeddings or local embeddings
- PyPDF
- LangChain or custom RAG logic

Project Architecture

```
graph TD; A[PDF Upload] --> B[Text Extraction]; B --> C[Chunking]; C --> D[Embedding Model]; D --> E[Vector Database]; E --> F[User Question]; F --> G[Retrieve Relevant Chunks]; G --> H[LLM Answer With Sources]; H --> I[Streamlit Chat UI];
```

MVP Scope

- Upload PDF
- Extract text
- Chunk document
- Store embeddings
- Ask questions
- Return answers with source passages

Stretch Goals

- Multi-document search
- Conversation memory
- PDF page citations
- Local LLM version
- Evaluation set with expected answers
- Persistent document library

GitHub Repository Structure

```
smart-doc-qa/  
├── app.py  
├── src/  
│   ├── ingestion.py  
│   ├── chunking.py  
│   ├── embeddings.py  
│   ├── retriever.py  
│   ├── qa_chain.py  
│   └── citations.py  
├── prompts/  
│   └── rag_answer.txt  
├── sample_docs/  
├── tests/  
├── requirements.txt  
└── README.md
```

Portfolio Description

Built a RAG-based document Q&A system that allows users to upload PDFs, ask natural language questions, and receive grounded answers with source references.

Resume Bullet Points

- Developed a retrieval-augmented generation system using embeddings, vector search, and LLMs to answer questions over uploaded documents with source citations.
- Implemented text chunking, vector storage, semantic retrieval, and prompt grounding to improve answer reliability and reduce hallucinations.

YouTube Video Ideas

- I Built a ChatGPT for My Own Documents
- RAG Explained by Building It From Scratch
- How Vector Databases Actually Work

TikTok / Shorts Ideas

- I asked my PDF a question and it answered
- RAG explained in 60 seconds
- What is an embedding?
- Why AI needs retrieval
- My PDF became searchable with AI

Estimated Difficulty

Beginner+ to Intermediate

Estimated Completion Time

3–4 weeks

Deployment Recommendation

Local first. Deploy demo version only with sample documents.

Project 3 — AI Data Insight Assistant

Problem Being Solved

Non-technical users often have CSV files but do not know how to analyze them.

This tool lets users upload a CSV and ask questions in natural language.

Technical Skills Learned

- CSV parsing
- Pandas
- exploratory data analysis
- LLM-assisted code generation
- chart generation
- safe execution patterns
- data profiling

Technologies Used

- Python
- Streamlit
- Pandas
- Plotly
- OpenAI or Claude API
- SQLite

Project Architecture

```
graph TD
    A[CSV Upload] --> B[Data Profiler]
    B --> C[Schema Summary]
    C --> D[User Question]
    D --> E[ ]
```

LLM Generates Analysis Plan

↓

Safe Pandas Execution

↓

Chart + Written Insight

MVP Scope

- Upload CSV
- Display dataset summary
- Ask natural language questions
- Generate charts
- Generate written insights
- Export analysis summary

Stretch Goals

- Auto-clean missing values
- Suggest questions
- Generate full EDA report
- Add anomaly detection
- Add forecasting
- Add SQL mode

GitHub Repository Structure

```
ai-data-insight-assistant/  
├─ app.py  
├─ src/  
│   ├─ data_loader.py  
│   ├─ profiler.py  
│   ├─ question_parser.py  
│   ├─ analyzer.py  
│   ├─ charts.py  
│   └─ report_generator.py  
├─ prompts/  
├─ sample_data/  
├─ tests/  
├─ requirements.txt  
└─ README.md
```

Portfolio Description

Built an AI-powered data analysis assistant that allows users to upload CSV files, ask natural language questions, generate charts, and receive plain-English business insights.

Resume Bullet Points

- Built an AI analytics assistant using Python, Pandas, Plotly, and LLMs to convert natural language questions into data analysis workflows.
- Designed a Streamlit dashboard for automated dataset profiling, visualization, and business insight generation.

YouTube Video Ideas

- I Built an AI Data Analyst
- Upload a CSV and Let AI Analyze It
- Building a Data App With Python and Streamlit

TikTok / Shorts Ideas

- AI analyzed my CSV in seconds
- This tool turns spreadsheets into insights
- Pandas plus AI is crazy
- I built a mini data analyst
- CSV analysis without writing code

Estimated Difficulty

Intermediate

Estimated Completion Time

3–4 weeks

Deployment Recommendation

Deploy with sample datasets only.

PHASE 2 — AUTOMATION

Project 4 — AI Content Repurposing Pipeline

Problem Being Solved

A single project can become many content pieces, but manually turning it into YouTube descriptions, TikTok hooks, LinkedIn posts, and summaries takes too long.

This project automates your Build With AI content workflow.

Technical Skills Learned

- transcription
- prompt chaining
- async workflows
- FastAPI basics
- background jobs
- content formatting
- file uploads

Technologies Used

- Python
- FastAPI
- Streamlit or simple frontend
- Whisper API
- OpenAI or Claude API
- PostgreSQL
- Celery or RQ
- Redis

Project Architecture

```
Video / Audio Upload
↓
Transcription
↓
Transcript Cleaner
↓
Prompt Chain
├── YouTube Title
├── YouTube Description
├── LinkedIn Post
├── 5 TikTok Hooks
└── Portfolio Notes
↓
PostgreSQL Storage
↓
Dashboard / Export
```

MVP Scope

- Upload audio or paste transcript
- Generate:
 - YouTube title
 - YouTube description
 - 5 TikTok ideas

- LinkedIn post
- Save previous generations
- Export Markdown

Stretch Goals

- Auto-generate timestamps
- Clip detection
- Scheduled posting reminders
- Brand voice settings
- Multi-project content library

GitHub Repository Structure

```
content-repurposing-pipeline/  
├── backend/  
│   ├── main.py  
│   ├── routes/  
│   ├── services/  
│   │   ├── transcriber.py  
│   │   ├── content_generator.py  
│   │   └── storage.py  
│   └── models/  
├── frontend/  
│   └── app.py  
├── prompts/  
├── docker-compose.yml  
├── requirements.txt  
└── README.md
```

Portfolio Description

Built an AI content repurposing pipeline that transforms project videos or transcripts into platform-specific content assets, including YouTube metadata, LinkedIn posts, and short-form video hooks.

Resume Bullet Points

- Designed a multi-step AI content pipeline using FastAPI, transcription, prompt chaining, and PostgreSQL storage to automate cross-platform publishing assets.
- Implemented reusable prompt templates and export workflows to reduce manual content preparation time.

YouTube Video Ideas

- I Built the Tool That Makes My Build With AI Content
- One Project Into 10 Pieces of Content With AI
- Building an AI Content Pipeline With FastAPI

TikTok / Shorts Ideas

- One video became 5 TikToks
- AI wrote my YouTube description
- My content workflow is automated
- Prompt chains explained
- FastAPI for AI workflows

Estimated Difficulty

Intermediate

Estimated Completion Time

3–5 weeks

Deployment Recommendation

Deploy after MVP works locally.

Project 5 — AI Job Hunter CRM

Problem Being Solved

Job seekers waste time tracking applications, comparing jobs, tailoring resumes, and preparing for interviews.

This project becomes a job-search operating system.

Technical Skills Learned

- database design
- CRUD apps
- job parsing
- embeddings
- semantic similarity
- resume-job matching
- email digest generation
- Kanban UI

Technologies Used

- Python
- Streamlit
- FastAPI
- PostgreSQL

- SQLAlchemy
- OpenAI or Claude API
- ChromaDB or pgvector

Project Architecture

```
Job Posting Input
↓
Job Parser
↓
Database Storage
↓
Resume Profile Embedding
↓
Job Match Scoring
↓
AI Tailoring Assistant
↓
Kanban Tracker
↓
Interview Prep Generator
```

MVP Scope

- Manually paste job posting
- Extract title, company, skills, salary, location
- Store in database
- Score against your resume/profile
- Generate tailored resume bullets
- Generate cover letter draft
- Track status: saved, applied, interview, rejected, offer

Stretch Goals

- Browser extension
- Daily job scraping
- Email digest
- Interview question generator
- Resume version history
- Company research agent
- Automated application analytics

GitHub Repository Structure

```
ai-job-hunter-crm/
├─ app.py
```

```
├── backend/
│   ├── main.py
│   ├── models.py
│   ├── database.py
│   └── routes/
├── src/
│   ├── job_parser.py
│   ├── matcher.py
│   ├── resume_tailor.py
│   ├── interview_prep.py
│   └── company_research.py
├── prompts/
├── migrations/
├── tests/
├── docker-compose.yml
└── README.md
```

Portfolio Description

Built an AI-powered job search CRM that parses job descriptions, scores fit against a candidate profile, generates tailored application materials, and tracks applications through a Kanban workflow.

Resume Bullet Points

- Built an AI job search platform using PostgreSQL, embeddings, and LLMs to parse job descriptions, score candidate fit, and generate tailored application materials.
- Designed a full-stack application workflow with status tracking, semantic matching, and interview preparation generation.

YouTube Video Ideas

- I Built an AI Job Hunter to Help My Job Search
- Can AI Match Me to Better Jobs?
- Building a Job Search CRM With Python

TikTok / Shorts Ideas

- AI scored this job against my resume
- My job tracker writes cover letters
- This tool finds missing resume keywords
- I built a CRM for job searching
- AI generated interview questions from a job post

Estimated Difficulty

Intermediate

Estimated Completion Time

4–6 weeks

Deployment Recommendation

Local first because it contains personal data. Deploy demo version with fake data.

Project 6 — AI Email Triage Assistant

Problem Being Solved

Inbox overload makes it hard to identify important emails, respond quickly, and organize communication.

This assistant classifies emails, prioritizes them, and drafts responses.

Technical Skills Learned

- Gmail API
- OAuth
- email parsing
- classification
- few-shot prompting
- privacy-aware design
- draft generation
- daily digest automation

Technologies Used

- Python
- Gmail API
- Streamlit
- SQLite
- OpenAI or Claude API
- APScheduler

Project Architecture

```
Gmail API
  ↓
Email Fetcher
  ↓
Email Cleaner
  ↓
LLM Classifier
```

```
↓  
Priority Scorer  
↓  
Draft Reply Generator  
↓  
Daily Digest Dashboard
```

MVP Scope

- Fetch latest 50 emails
- Categorize emails
- Score urgency
- Generate daily digest
- Draft suggested replies
- Mark emails as action needed / later / ignore

Stretch Goals

- Learn user writing style
- Calendar detection
- Auto-label emails
- Weekly inbox report
- Slack/Discord urgent alert
- Local-only privacy mode

GitHub Repository Structure

```
ai-email-triage/  
├─ app.py  
├─ src/  
│   ├─ gmail_client.py  
│   ├─ classifier.py  
│   ├─ priority_scorer.py  
│   ├─ reply_generator.py  
│   ├─ digest.py  
│   └─ database.py  
├─ prompts/  
├─ tests/  
├─ .env.example  
└─ README.md
```

Portfolio Description

Built an AI email assistant that connects to Gmail, classifies incoming messages, prioritizes urgent items, generates daily digests, and drafts suggested replies.

Resume Bullet Points

- Integrated Gmail API with an AI classification pipeline to categorize and prioritize emails based on urgency, sender, and content.
- Built a privacy-conscious email assistant with daily summaries, reply drafting, and local data storage.

YouTube Video Ideas

- I Gave AI My Inbox and Built an Email Assistant
- Building a Gmail AI Assistant With Python
- AI Email Triage From Scratch

TikTok / Shorts Ideas

- AI found my most important emails
- My inbox now summarizes itself
- Gmail API in 60 seconds
- I taught AI to draft replies
- Inbox zero with Python

Estimated Difficulty

Intermediate

Estimated Completion Time

3–5 weeks

Deployment Recommendation

Local only.

PHASE 3 — DATA & INTELLIGENCE

Project 7 — Market / Business Intelligence Monitor

Problem Being Solved

Businesses need to monitor competitors, trends, news, and market signals, but manual research is slow.

This project tracks sources and turns them into structured intelligence reports.

Technical Skills Learned

- APIs
- web scraping

- RSS feeds
- scheduled data pipelines
- text classification
- sentiment analysis
- entity extraction
- dashboards

Technologies Used

- Python
- FastAPI
- PostgreSQL
- RSS / News APIs
- BeautifulSoup
- Prefect
- Streamlit
- Plotly
- OpenAI or Claude API

Project Architecture

```
Sources:
- RSS
- News API
- Websites
- Reddit
    ↓
Scheduled Pipeline
    ↓
Text Cleaner
    ↓
Entity + Sentiment Extraction
    ↓
PostgreSQL
    ↓
Dashboard
    ↓
Weekly Intelligence Report
```

MVP Scope

- Track 5 companies or topics
- Pull daily news/RSS updates
- Summarize important updates
- Classify by topic
- Display dashboard
- Generate weekly report

Stretch Goals

- Competitive landscape map
- sentiment trend chart
- alert system
- source credibility scoring
- PDF report export
- company comparison mode

GitHub Repository Structure

```
market-intelligence-monitor/  
├── backend/  
│   ├── main.py  
│   ├── pipelines/  
│   ├── services/  
│   └── models/  
├── dashboard/  
│   └── app.py  
├── prompts/  
├── docker-compose.yml  
├── tests/  
└── README.md
```

Portfolio Description

Built a business intelligence monitoring system that collects external market signals, classifies trends, extracts key entities, and generates structured intelligence reports.

Resume Bullet Points

- Built an automated market intelligence pipeline using Python, PostgreSQL, scheduled ingestion, LLM summarization, and dashboard visualizations.
- Designed a reporting workflow that transforms unstructured news and web data into structured business insights.

YouTube Video Ideas

- I Built an AI That Tracks Market Trends
- Building a Business Intelligence Agent From Scratch
- How Companies Can Use AI for Competitor Research

TikTok / Shorts Ideas

- AI tracked competitors for me
- My dashboard reads the news
- Business intelligence with Python

- I built a trend monitor
- AI summarized a week of market news

Estimated Difficulty

Intermediate+

Estimated Completion Time

5–6 weeks

Deployment Recommendation

Deploy demo version with public topics.

Project 8 — ML Churn Prediction API

Problem Being Solved

Companies lose revenue when customers churn. This project predicts which customers are likely to leave and explains why.

This is your first serious ML engineering project.

Technical Skills Learned

- feature engineering
- classification models
- model evaluation
- XGBoost
- SHAP explainability
- MLflow
- FastAPI model serving
- Docker

Technologies Used

- Python
- Scikit-learn
- XGBoost
- MLflow
- SHAP
- FastAPI
- Docker
- PostgreSQL

Project Architecture

```
Raw Customer Data
↓
EDA
↓
Feature Engineering
↓
Model Training
↓
MLflow Experiment Tracking
↓
Best Model Selection
↓
SHAP Explanation
↓
FastAPI Prediction Endpoint
↓
Streamlit Demo Dashboard
```

MVP Scope

- Use public churn dataset
- Train baseline model
- Train XGBoost model
- Track experiments with MLflow
- Create `/predict` API endpoint
- Show prediction and top reasons

Stretch Goals

- retraining pipeline
- drift detection
- batch predictions
- model monitoring
- synthetic data generator
- Docker deployment

GitHub Repository Structure

```
ml-churn-prediction-api/
├─ notebooks/
│   └─ 01_eda.ipynb
├─ src/
│   └─ features.py
│   └─ train.py
```

```
|   |— evaluate.py
|   |— explain.py
|   |— predict.py
|— api/
|   |— main.py
|— dashboard/
|   |— app.py
|— models/
|— tests/
|— Dockerfile
|— requirements.txt
|— README.md
```

Portfolio Description

Built an end-to-end ML churn prediction system with feature engineering, model training, experiment tracking, explainability, and production-style API deployment.

Resume Bullet Points

- Developed an end-to-end churn prediction system using XGBoost, MLflow, SHAP, FastAPI, and Docker.
- Built a model-serving API that returns churn probability and explainable feature-level reasons for each prediction.

YouTube Video Ideas

- Full Machine Learning Project From Data to API
- I Built an ML Model That Predicts Customer Churn
- MLflow and SHAP Explained Through a Real Project

TikTok / Shorts Ideas

- My ML model predicts customer churn
- SHAP values explained in 60 seconds
- MLflow changed my workflow
- Turning a model into an API
- XGBoost vs baseline model

Estimated Difficulty

Intermediate+

Estimated Completion Time

5–7 weeks

Deployment Recommendation

Deploy API and demo dashboard.

Project 9 — Natural Language to SQL Analytics Engine

Problem Being Solved

Business users often need answers from databases but cannot write SQL.

This tool converts plain English questions into safe SQL queries and visualized answers.

Technical Skills Learned

- text-to-SQL
- SQL safety
- schema introspection
- PostgreSQL
- query validation
- chart generation
- analytics UX

Technologies Used

- Python
- FastAPI
- PostgreSQL
- SQLAlchemy
- Streamlit
- Plotly
- OpenAI or Claude API

Project Architecture

```
graph TD;
    A[Database Schema] --> B[Schema Summarizer];
    B --> C[User Question];
    C --> D[SQL Generator];
    D --> E[SQL Validator];
    E --> F[Read-Only Execution];
```

```
↓  
Result Table  
↓  
Chart Generator  
↓  
AI Explanation
```

MVP Scope

- Connect to sample PostgreSQL database
- Ask plain-English questions
- Generate SQL
- Validate read-only queries
- Display table results
- Generate simple chart and explanation

Stretch Goals

- saved queries
- multi-table joins
- dashboard builder
- correction loop
- export CSV
- role-based permissions

GitHub Repository Structure

```
nl-to-sql-engine/  
├── backend/  
│   ├── main.py  
│   ├── db.py  
│   ├── schema_reader.py  
│   ├── sql_generator.py  
│   ├── sql_validator.py  
│   └── query_runner.py  
├── frontend/  
│   └── app.py  
├── sample_db/  
├── prompts/  
├── tests/  
└── README.md
```

Portfolio Description

Built a natural language analytics engine that translates business questions into safe SQL queries, executes them against a database, and returns visualized insights.

Resume Bullet Points

- Built a text-to-SQL analytics engine using LLMs, PostgreSQL, query validation, and visualization tools.
- Implemented schema-aware prompting and read-only SQL execution to safely answer business questions from relational data.

YouTube Video Ideas

- I Built an AI That Writes SQL
- The Future of Business Intelligence
- Text-to-SQL With Claude and PostgreSQL

TikTok / Shorts Ideas

- I asked my database a question in English
- AI wrote the SQL for me
- Text-to-SQL explained
- This could replace basic BI queries
- PostgreSQL plus AI is powerful

Estimated Difficulty

Intermediate+

Estimated Completion Time

4–6 weeks

Deployment Recommendation

Deploy with sample database only.

PHASE 4 — AI AGENTS

Project 10 — Autonomous Research Agent

Problem Being Solved

Deep research requires searching, reading, filtering, comparing, and summarizing multiple sources.

This agent automates the research workflow.

Technical Skills Learned

- LangGraph
- agent state

- planning
- tool calling
- web search tools
- source extraction
- memory
- report generation
- evaluation

Technologies Used

- Python
- LangGraph
- FastAPI
- Streamlit
- Tavily or search API
- ChromaDB
- OpenAI or Claude API
- PostgreSQL

Project Architecture

```
graph TD;
    A[Research Question] --> B[Planner Agent];
    B --> C[Search Tool];
    C --> D[Reader Tool];
    D --> E[Extractor Agent];
    E --> F[Memory Store];
    F --> G[Critic Agent];
    G --> H[Report Writer Agent];
    H --> I[Cited Research Report];
```

MVP Scope

- Input research question
- Generate research plan
- Search web
- Extract key points
- Produce structured report
- Include source links

- Show agent steps

Stretch Goals

- critic/reviewer agent
- PDF export
- saved research sessions
- source reliability score
- comparison tables
- domain-specific research modes

GitHub Repository Structure

```
autonomous-research-agent/  
├── agent/  
│   ├── graph.py  
│   ├── state.py  
│   ├── nodes/  
│   │   ├── planner.py  
│   │   ├── searcher.py  
│   │   ├── reader.py  
│   │   ├── extractor.py  
│   │   ├── critic.py  
│   │   └── writer.py  
│   └── tools/  
├── api/  
├── frontend/  
├── memory/  
├── prompts/  
├── tests/  
└── README.md
```

Portfolio Description

Built an autonomous research agent that plans research tasks, searches external sources, extracts key information, critiques findings, and generates structured reports with citations.

Resume Bullet Points

- Architected a LangGraph-based autonomous research agent with planning, search, extraction, critique, memory, and report-generation nodes.
- Implemented tool-calling workflows and persistent research memory to support multi-step knowledge work automation.

YouTube Video Ideas

- I Built an AI Research Agent

- LangGraph Explained by Building a Real Agent
- Chatbot vs Agent: What Is the Difference?

TikTok / Shorts Ideas

- AI researched this topic for me
- Watch an agent use tools
- LangGraph in 60 seconds
- AI agents are not just chatbots
- My research workflow is automated

Estimated Difficulty

Advanced

Estimated Completion Time

6–8 weeks

Deployment Recommendation

Deploy demo version.

Project 11 — Customer Support Agent System

Problem Being Solved

Businesses receive repetitive support questions. A support agent system can classify issues, answer from a knowledge base, and escalate complex cases.

Technical Skills Learned

- multi-agent systems
- RAG knowledge base
- ticket classification
- escalation logic
- CRM-style database design
- API design
- evaluation of AI responses

Technologies Used

- Python
- FastAPI
- PostgreSQL
- ChromaDB or pgvector
- LangGraph

- Streamlit or React
- Docker

Project Architecture

```
Customer Message
↓
Triage Agent
↓
Category Router
├─ Billing Agent
├─ Technical Agent
├─ Account Agent
└─ Escalation Agent
↓
Knowledge Base Retrieval
↓
Response Generator
↓
Confidence Check
↓
Human Escalation or Final Reply
```

MVP Scope

- Submit support ticket
- Classify ticket type
- Retrieve relevant knowledge base articles
- Generate suggested reply
- Escalate low-confidence cases
- Store tickets in database

Stretch Goals

- admin dashboard
- ticket analytics
- response quality evaluation
- user feedback loop
- Slack integration
- multi-tenant support

GitHub Repository Structure

```
customer-support-agent-system/
├─ backend/
│   └─ main.py
```

```
|   |   | models.py
|   |   | routes/
|   |   | agents/
|   |   |   | triage.py
|   |   |   | billing.py
|   |   |   | technical.py
|   |   |   | escalation.py
|   |   | retrieval/
|   | frontend/
|   | knowledge_base/
|   | tests/
|   | docker-compose.yml
|   | README.md
```

Portfolio Description

Built a multi-agent customer support system that classifies tickets, retrieves relevant knowledge base content, drafts responses, and escalates uncertain cases to humans.

Resume Bullet Points

- Built a multi-agent support automation system using LangGraph, FastAPI, PostgreSQL, and vector search for ticket classification and response generation.
- Designed escalation logic and confidence thresholds to support human-in-the-loop AI customer service workflows.

YouTube Video Ideas

- I Built an AI Customer Support Team
- Multi-Agent Systems Explained With a Real Business Problem
- Building a Support Agent With RAG

TikTok / Shorts Ideas

- AI handled this support ticket
- Multi-agent system in 60 seconds
- My AI support team has roles
- RAG for customer service
- Human escalation matters

Estimated Difficulty

Advanced

Estimated Completion Time

6–8 weeks

Deployment Recommendation

Deploy with fake company data.

Project 12 — AI Code Review / PR Assistant

Problem Being Solved

Developers spend time reviewing code for bugs, style issues, documentation gaps, and security concerns.

This assistant reviews pull requests and generates structured feedback.

Technical Skills Learned

- GitHub webhooks
- pull request APIs
- code diff parsing
- static analysis
- LLM code review
- CI/CD integration
- developer tooling

Technologies Used

- Python
- FastAPI
- GitHub API
- GitHub Actions
- OpenAI or Claude API
- Docker
- PostgreSQL

Project Architecture

```
graph TD
    A[GitHub Pull Request] --> B[Webhook Trigger]
    B --> C[Diff Fetcher]
    C --> D[Static Checks]
    D --> E[LLM Code Reviewer]
    E --> F[Feedback Formatter]
```

↓
Comment on PR
↓
Store Review History

MVP Scope

- Connect to GitHub repo
- Fetch PR diff
- Generate code review comments
- Summarize PR changes
- Identify risks and missing tests

Stretch Goals

- GitHub Action integration
- security review mode
- style guide configuration
- automatic test suggestions
- repo-specific memory
- dashboard of review history

GitHub Repository Structure

```
ai-code-review-assistant/  
├── api/  
│   ├── main.py  
│   ├── webhooks.py  
│   └── github_client.py  
├── reviewer/  
│   ├── diff_parser.py  
│   ├── static_checks.py  
│   ├── llm_reviewer.py  
│   └── formatter.py  
├── prompts/  
├── tests/  
├── Dockerfile  
└── README.md
```

Portfolio Description

Built an AI-powered code review assistant that integrates with GitHub, analyzes pull request diffs, and generates structured feedback on bugs, missing tests, and code quality.

Resume Bullet Points

- Built a GitHub-integrated AI code review assistant using webhooks, diff parsing, LLM review prompts, and automated PR feedback.
- Implemented developer workflow automation with FastAPI, GitHub API, Docker, and CI/CD-ready architecture.

YouTube Video Ideas

- I Built an AI That Reviews My Code
- Building a GitHub PR Bot With Python
- Can AI Catch Bugs in My Pull Requests?

TikTok / Shorts Ideas

- AI reviewed my pull request
- GitHub webhook in 60 seconds
- My code reviewer is automated
- AI found a missing test
- Building dev tools with AI

Estimated Difficulty

Advanced

Estimated Completion Time

5–7 weeks

Deployment Recommendation

Deploy as a GitHub App or webhook service.

PHASE 5 — PRODUCTION-READY SYSTEMS

Project 13 — Full-Stack AI SaaS Starter

Problem Being Solved

Many AI projects stay as demos. This project teaches how to turn an AI tool into a real product with users, authentication, subscriptions, and deployment.

Technical Skills Learned

- full-stack architecture
- authentication

- user accounts
- payments
- API rate limits
- database migrations
- logging
- deployment
- CI/CD

Technologies Used

- FastAPI
- PostgreSQL
- SQLAlchemy
- React or Next.js
- Stripe
- Docker
- GitHub Actions
- Render / Railway / Fly.io / AWS

Project Architecture

```
graph TD; FrontendApp[Frontend App] --> AuthSystem[Auth System]; AuthSystem --> FastAPIBackend[FastAPI Backend]; FastAPIBackend --> PostgreSQL[PostgreSQL]; PostgreSQL --> AIServiceLayer[AI Service Layer]; AIServiceLayer --> UsageTracking[Usage Tracking]; UsageTracking --> StripeBilling[Stripe Billing]; StripeBilling --> LogsMonitoring[Logs + Monitoring];
```

MVP Scope

- user signup/login
- dashboard
- one AI feature
- usage limits
- PostgreSQL database
- deployed backend and frontend

Stretch Goals

- Stripe subscription
- admin dashboard
- email notifications
- team workspaces
- API keys
- monitoring dashboard

GitHub Repository Structure

```
ai-saas-starter/  
├── backend/  
│   ├── main.py  
│   ├── auth/  
│   ├── users/  
│   ├── billing/  
│   ├── ai_services/  
│   └── database/  
├── frontend/  
│   ├── pages/  
│   ├── components/  
│   └── lib/  
├── infra/  
├── tests/  
├── docker-compose.yml  
└── README.md
```

Portfolio Description

Built a production-style AI SaaS starter platform with authentication, user accounts, AI feature usage, PostgreSQL storage, and deployment-ready architecture.

Resume Bullet Points

- Built a full-stack AI SaaS application with FastAPI, PostgreSQL, authentication, usage tracking, and deployed frontend/backend services.
- Implemented production-oriented software patterns including environment management, database migrations, API design, and CI/CD workflows.

YouTube Video Ideas

- Turning an AI Project Into a SaaS
- Building My First Production AI App
- AI SaaS Architecture Explained

TikTok / Shorts Ideas

- My AI demo became a SaaS
- Auth is harder than AI
- What every AI app needs
- Stripe plus AI app
- Production-ready AI app checklist

Estimated Difficulty

Advanced+

Estimated Completion Time

8–10 weeks

Deployment Recommendation

Deploy fully.

Project 14 — Visual Automation Platform

Problem Being Solved

Businesses need to connect tools, run automations, and build workflows, but tools like Zapier can be expensive or limited.

This becomes your capstone project.

Technical Skills Learned

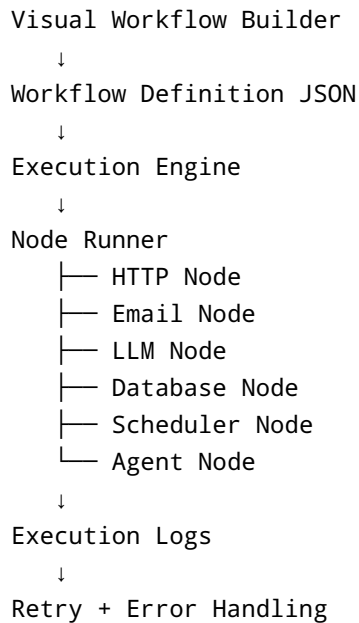
- workflow engines
- DAG execution
- queues
- retries
- webhooks
- visual builders
- agent nodes
- background processing
- product architecture

Technologies Used

- FastAPI
- React
- React Flow

- PostgreSQL
- Redis
- Celery
- Docker
- LangGraph
- OpenAI or Claude API

Project Architecture



MVP Scope

- visual workflow builder
- 5 node types
- manual trigger
- webhook trigger
- execution logs
- error handling
- save workflows

Stretch Goals

- cron scheduling
- user accounts
- workspace isolation
- AI agent node
- plugin marketplace
- cloud-hosted version

GitHub Repository Structure

```
visual-automation-platform/  
├── backend/  
│   ├── main.py  
│   ├── workflows/  
│   ├── nodes/  
│   ├── execution_engine/  
│   ├── scheduler/  
│   └── logs/  
├── frontend/  
│   ├── components/  
│   ├── workflow_editor/  
│   └── pages/  
├── workers/  
├── infra/  
├── tests/  
├── docker-compose.yml  
└── README.md
```

Portfolio Description

Built a visual automation platform that allows users to design workflows, connect nodes, trigger automations, run background jobs, and monitor execution history.

Resume Bullet Points

- Architected a workflow automation platform using FastAPI, React Flow, PostgreSQL, Redis, Celery, and Docker.
- Built a node-based execution engine supporting webhooks, manual triggers, LLM nodes, database actions, and error handling.

YouTube Video Ideas

- I'm Building My Own Zapier With AI
- Designing a Workflow Automation Engine
- Building a Visual Automation Platform From Scratch

TikTok / Shorts Ideas

- I built my own Zapier
- Workflow engines explained
- Drag-and-drop automation builder
- AI agents inside workflows
- This is my biggest project yet

Estimated Difficulty

Advanced+

Estimated Completion Time

10–16 weeks

Deployment Recommendation

Deploy as capstone product.

Technology Decision Guide

Streamlit

Use for:

- fast demos
- dashboards
- internal tools
- project MVPs
- portfolio demos

Best projects:

- Build Log AI
- Document Q&A
- Data Insight Assistant
- Job Hunter MVP
- Market Intelligence Dashboard
- Research Agent demo

FastAPI

Use when:

- you need real API endpoints
- frontend/backend are separate
- you need webhooks
- background jobs call APIs
- project feels like real software

Best projects:

- Content Pipeline
- Job Hunter CRM
- Churn Prediction API
- NL to SQL Engine
- Support Agent System
- Code Review Assistant
- AI SaaS Starter
- Automation Platform

PostgreSQL

Use when:

- the project has relationships
- the project stores users, jobs, reports, tickets, workflows
- you may deploy it later
- you want realistic backend experience

Best projects:

- Job Hunter CRM
- Content Pipeline
- Market Intelligence Monitor
- Churn Prediction API
- NL to SQL Engine
- Support Agent System
- SaaS Starter
- Automation Platform

SQLite

Use when:

- local-only
- prototype
- single-user
- no deployment needed

Best projects:

- Build Log AI
- Email Triage Assistant local mode
- early prototypes

Vector Databases

Use when:

- you need semantic search
- documents need retrieval
- knowledge base is required
- AI needs memory

Best projects:

- Document Q&A
- Job Hunter resume matching
- Market Intelligence Monitor
- Research Agent
- Support Agent System

Recommended:

- ChromaDB for local learning
- FAISS for simple local search
- pgvector when using PostgreSQL in production

LangGraph / Agent Frameworks

Use when:

- system has multiple steps
- AI must plan
- tools are involved
- there are decision branches
- agents have different roles

Best projects:

- Research Agent
- Support Agent System
- Code Review Assistant
- Automation Platform

Do not use LangGraph too early.

First learn:

- API calls
- database storage
- RAG
- FastAPI

- background jobs

Then use LangGraph.

Deployment Guide

Keep Local

- Email Triage Assistant
- personal job tracker version
- finance/personal data tools
- any project using private documents

Deploy Demo Version

- Build Log AI
- Document Q&A with sample docs
- Data Insight Assistant with sample CSVs
- Job Hunter with fake job data
- Market Intelligence Monitor with public topics
- Research Agent with limited search

Fully Deploy

- Churn Prediction API
 - NL to SQL demo
 - SaaS Starter
 - Automation Platform
 - Code Review Assistant
-

Weekly Build Workflow

This is the repeatable system.

Day 1 — Define

Deliverables:

- problem statement
- user persona
- manual workflow being replaced
- project scope

- GitHub repo created
- README draft

Questions:

- Who is this for?
- What problem does it solve?
- What does the MVP need?
- What can wait?

Day 2 — Architecture

Deliverables:

- architecture diagram
- folder structure
- database schema if needed
- API design if needed
- prompt templates

Questions:

- What are the main components?
- What data flows through the system?
- Where can it fail?
- What should be logged?

Day 3-5 — Build MVP

Deliverables:

- working core feature
- clean commits
- basic UI
- basic tests
- sample data
- screenshots

Rule:

Do not polish before the core workflow works.

Day 6 — Package

Deliverables:

- README finished

- demo GIF or screenshots
- portfolio write-up
- resume bullets
- lessons learned
- known limitations

README structure:

```
# Project Name

## Problem

## Solution

## Demo

## Architecture

## Tech Stack

## Features

## How to Run

## What I Learned

## Limitations

## Future Improvements
```

Day 7 — Content

Deliverables:

- 1 YouTube video
 - 5 TikTok/Shorts
 - 1 LinkedIn post
 - GitHub pinned repo
 - portfolio updated
-

YouTube Video Template

Title Format

Use outcome-based titles.

Bad:

- Building a LangGraph App
- FastAPI Project Tutorial

Good:

- I Built an AI That Reads My Documents
- I Built an AI Job Hunter to Help My Job Search
- I Built a Customer Support AI Team

Video Structure

```
0:00 – Show final result
0:20 – Problem
1:00 – Architecture
2:00 – Build process
6:00 – Demo
8:00 – What broke
9:00 – What I learned
10:00 – What I would improve
```

Your Style

Do not act like a guru.

Position yourself as:

“I’m learning by building real systems. Here’s what worked, what broke, and what I learned.”

That is more trustworthy.

TikTok / Shorts Template

Every project should create 5 short videos:

Short 1 — Final Demo

"Here's what I built."

Short 2 — Problem

"This annoying problem is why I built it."

Short 3 — Technical Concept

"One concept I learned from this project."

Short 4 — Mistake / Bug

"This broke and here's how I fixed it."

Short 5 — Lesson

"What I learned after building this."

LinkedIn Post Template

I just finished building [project name].

The problem:

[1-2 lines about the real-world problem]

What I built:

[short explanation]

Tech stack:

[Python, FastAPI, PostgreSQL, etc.]

What I learned:

[3 bullets]

Hardest part:

[honest technical challenge]

Next improvement:

[future feature]

GitHub:

[link]

Demo:

[link if available]

Portfolio Case Study Template

Project Name

Problem

What real-world workflow or business problem does this solve?

Solution

What did you build?

Tech Stack

What tools did you use?

Architecture

How does data move through the system?

Key Features

What can it do?

Challenges

What was hard?

Results

What changed after building it?

Future Improvements

What would you add next?

Resume Strategy

Do not add every project to your resume.

Use different project groups depending on job target.

For AI Engineer / Applied AI Roles

Highlight:

- Document Q&A RAG
- Research Agent
- Support Agent System
- AI SaaS Starter
- Code Review Assistant

For Data Scientist Roles

Highlight:

- AI Data Insight Assistant
- Market Intelligence Monitor
- Churn Prediction API
- NL to SQL Engine

For Data Engineer / Analytics Engineer Roles

Highlight:

- Market Intelligence Monitor
- NL to SQL Engine
- Job Hunter CRM
- Content Pipeline

For Backend Developer Roles

Highlight:

- Churn Prediction API
- Code Review Assistant
- SaaS Starter
- Automation Platform

For Automation / AI Solutions Roles

Highlight:

- Content Pipeline
 - Email Triage Assistant
 - Support Agent System
 - Automation Platform
-

Execution Rules

Rule 1 — Finish Before Expanding

Do not add stretch goals until MVP is complete.

Rule 2 — Every Project Needs a Demo

A project without a demo is invisible.

Rule 3 — Every Project Needs a README

A project without documentation looks unfinished.

Rule 4 — Measure Something

Even simple metrics count:

- time saved
- number of documents processed
- number of jobs analyzed
- model accuracy
- API response time
- number of sources summarized

Rule 5 — Do Not Fake Metrics

Only claim numbers you actually measured.

Rule 6 — Build in Public, But Learn in Private

You do not need to show every mistake.

Show:

- final result
- lessons
- process
- bugs worth sharing

Rule 7 — The First 3 Projects Matter Most

If you finish:

1. Build Log AI
2. Document Q&A
3. Data Insight Assistant

You will already have:

- API experience
- Streamlit experience
- LLM experience
- RAG experience
- GitHub proof
- portfolio pieces
- YouTube/TikTok content
- LinkedIn posts

That is the foundation.

First 90 Days

Month 1

Build Log AI

Goal:

Build your proof-of-work engine.

Output:

- 1 repo
- 1 demo
- 1 YouTube video
- 5 shorts
- 1 LinkedIn post

Month 2

Smart Document Q&A

Goal:

Learn RAG and vector search.

Output:

- RAG portfolio project
- demo with sample PDFs
- technical explainer video

Month 3

AI Data Insight Assistant

Goal:

Combine data analytics and AI.

Output:

- CSV analysis app
- portfolio-ready data/AI project
- video showing AI turning data into insights

First Project Execution Plan

Start with Build Log AI.

Milestone 1 — Repo and Setup

Tasks:

- create GitHub repo
- create virtual environment
- install Streamlit, requests, python-dotenv
- create folder structure
- add README

Commit message:

```
init project structure for Build Log AI
```

Milestone 2 — GitHub API Fetcher

Tasks:

- connect to GitHub API
- fetch repo commits
- parse commit messages
- display raw commit data

Commit message:

```
add GitHub commit fetching service
```

Milestone 3 — Activity Summarizer

Tasks:

- group commits by date
- clean commit messages
- create activity summary without AI first

Commit message:

```
add commit aggregation and activity summary
```

Milestone 4 — LLM Summary Generator

Tasks:

- create prompt templates
- call OpenAI or Claude API
- generate weekly summary
- generate LinkedIn post
- generate TikTok hooks

Commit message:

```
add AI-generated developer summaries
```

Milestone 5 — Streamlit Dashboard

Tasks:

- create simple UI
- input GitHub username/repo
- choose date range
- display generated outputs
- add export button

Commit message:

```
build Streamlit dashboard for AI summaries
```

Milestone 6 — Documentation and Demo

Tasks:

- finish README
- add screenshots
- add architecture diagram
- record demo
- write LinkedIn post
- create 5 shorts

Commit message:

```
add documentation and demo assets
```

Final Direction

The long-term mission is:

Build useful AI systems that solve real problems, document the process, and use every project as proof that you can work as an AI/data/software professional.

Do not rush to advanced agents.

Earn the advanced projects by shipping the foundation projects first.

The roadmap should make you progressively stronger in:

- software engineering
- AI application development
- automation
- data systems
- machine learning
- agent workflows
- production deployment
- business problem-solving

The channel is not the main product.

Your skill stack is the product.

Your GitHub is the evidence.

Your videos are the documentation.

Your portfolio is the sales page.

Build With AI is the operating system.